МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1
по курсу «Параллельные и распределенные вычисления»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами.

Выполнил: М. Д. Иванченко
Группа: М8О-310Б-23
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2026

Условие

Цель работы: ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений(CUDA).
Реализация одной из примитивных операций над векторами.
Вариант задания: вариант 2, вычитание векторов.
Входные данные. На первой строке задано число n – размер векторов. В следующих 2-х строках, записано по n вещественных чисел – элементы векторов.
Выходные данные. Необходимо вывести n чисел – результат вычитания исходных векторов.

Программное и аппаратное обеспечение

Дать характеристики графического процессора (compute capability, графическая память, разделяемая память, константная память, количество регистров на блок, максимальное количество блоков и нитей, количество мультипроцессоров), процессора, оперативной памяти и жесткого диска. Описать программное обеспечение (OS, IDE, compiler и тд.).
Графический процессор:

Характеристика	**Значение**
Compute Capability	8.6
Графическая память (Video Memory)	4 ГБ GDDR6
Шина памяти (Memory Interface)	128-бит
Количество мультипроцессоров (SM)	20
Количество ядер CUDA	2560
Количество тензорных ядер (Tensor Cores)	80 (3-го поколения)
Количество RT-ядер (Ray Tracing Cores)	20 (2-го поколения)
Разделяемая память (Shared Memory)	64 КБ
Константная память (Constant Memory)	64 КБ
Регистров на блок (Registers per Block)	255
Макс. количество нитей (Max Threads)	1536
Макс. количество блоков (Max Blocks)	16

Процессор:

Параметр	**Значение**
Модель	Intel Core i7-12650H (12-е поколение)
Всего ядер	10 (6 производительных + 4 энергоэффективных)
Всего потоков	16
Базовая частота	2.30 ГГц
Макс. частота Turbo	До 4.70 ГГц
Кэш L3 (Smart Cache)	24.0 МБ
Кэш L2	8.8 МБ
Кэш L1	800 КБ
Техпроцесс	Intel 7 (10 нм Enhanced)
Виртуализация	Включена (VT-x / VT-d)

Оперативная память: 32 ГБ.
Объем жёсткого диска: 954 ГБ.
OS: Windows 10.
IDE: Microsoft Visual Studio Community 2022.
Compiler: NVIDIA CUDA Compiler.

Метод решения

Программа реализует поэлементное вычитание двух векторов (C = A - B) с использованием графического процессора. Пользователь вводит размер массивов и их элементы, после чего данные копируются в видеопамять, на GPU запускается ядро для параллельного вычисления разности, а результат возвращается в оперативную память и выводится на экран.

Описание программы

В коде используются следующие основные типы данных и функции CUDA. Тип double обеспечивает хранение элементов векторов с двойной точностью, cudaError_t представляет перечисление всех возможных ошибок CUDA и используется для проверки успешности каждой операции. Функция cudaSetDevice(0) выбирает активное устройство GPU. cudaMalloc выделяет память на видеокарте под массивы dev_a, dev_b и dev_c. cudaMemcpy выполняет копирование данных между хостом и устройством с

параметрами cudaMemcpyHostToDevice (для отправки исходных векторов A и B на GPU) и cudaMemcpyDeviceToHost (для получения результата C обратно). cudaGetLastError и cudaGetErrorString позволяют получить и интерпретировать ошибки при запуске ядра, а cudaDeviceSynchronize ожидает завершения всех операций на GPU. cudaFree освобождает ранее выделенную видеопамять.

Реализованное ядро subKernel, помеченное спецификатором global, выполняется на GPU. Внутри ядра каждая нить вычисляет свой глобальный индекс по формуле blockIdx.x * blockDim.x + threadIdx.x, а затем с помощью паттерна grid-stride loop (цикл с шагом, равным общему количеству нитей blockDim.x * gridDim.x) обрабатывает все элементы массива, распределяя работу между нитями: пока индекс меньше размера массива, нить выполняет операцию c[index] = a[index] - b[index] и увеличивает индекс на шаг. Такой подход позволяет эффективно обрабатывать массивы произвольного размера, не требуя кратности количеству нитей, и обеспечивает равномерную загрузку вычислительных ресурсов GPU.

Результаты

Размер входного вектора	**Конфигурация**	**Время GPU (мс)**
10^3	<<1, 32>>	0.076
10^3	<<16, 32>>	0.065
10^3	<<32, 32>>	0.052
10^5	<<1, 32>>	1.638
10^5	<<256, 256>>	0.842
10^5	<<1024, 1024>>	0.099
10^7	<<1, 32>>	160.321
10^7	<<256, 256>>	2.353
10^7	<<1024, 1024>>	0.656
10^8	<<1, 32>>	286.587
10^8	<<256, 256>>	20.387
10^8	<<1024, 1024>>	2.36

Время CPU:

Размер вектора	Время (мс)
10^3	0.009
10^5	0.168
10^7	15.36
10^8	262.68

Выводы

Реализованный алгоритм поэлементного вычитания векторов является базовым шаблоном параллельных вычислений, применяемым в численном моделировании, обработке сигналов, машинном обучении и вычислительной линейной алгебре. При низкой сложности программирования (2-3 балла из 10) основными проблемами стали выбор оптимальной конфигурации запуска, корректное управление памятью при ошибках и учет асинхронности выполнения. На основе сравнения и анализа результатов можно заметить, что для малых размеров GPU уступает CPU из-за

накладных расходов, однако при оптимальных конфигурациях и увеличении количества входных данных GPU сильно обгоняет CPU по времени, что подтверждает эффективность использования GPU для массовой параллельной обработки данных.

Реализованный алгоритм поэлементного вычитания векторов является базовым шаблоном параллельных вычислений, применяемым в численном моделировании для обработки сеточных функций при решении дифференциальных уравнений. В обработке сигналов и изображений он используется для реализации фильтров высоких частот, выделения контуров и вычитания фона в видеопотоках. В машинном обучении алгоритм лежит в основе вычисления градиентов при обратном распространении ошибки и обновлении весов нейронных сетей. Кроме того, он применяется в вычислительной линейной алгебре (метод сопряженных градиентов, итерационные методы), финансовом моделировании и научных расчетах в молекулярной динамике и астрофизике.